Open in app ↗ Sign up Sign In

Published in Towards Data Science

Thomas Albin Follow

May 24, 2020 · 7 min read · ✨ · Listen

Save

SPACE SCIENCE WITH PYTHON

Space Science with Python — Supplements for Papers

Part 10 of the tutorial series is a Supplementary Article on Supplementary Materials. How can we visualise multi-dimensional functions properly?

26

Comet McNaught and its tails in the year 2007. The front hill with the telescope domes is Mount Paranal in Chile. Credit: S. Deiries/ESO; License: Creative Commons Attribution 4.0 International License

Preface

This is the 10th part of my Python tutorial series "Space Science with Python". All codes that are shown here are uploaded on GitHub. Enjoy!

Today's lesson is a "Supplementary Article" on how to create "Supplementary Material" for papers. Supplementary materials can be anything like tables, datasets etc. that cannot fit in a scientific paper. Here, we will have a look at the Tisserand Parameter and how to create an animation of a multi-dimensional function.

Introduction

In our last sessions, we learned about comets, their origin and their orbital attributes. Last time we found that an astrodynamical theory by Félix Tisserand [1] from the 19th century helps us to describe and explain the dynamical link between periodic (P type) comets and Jupiter. The equation returns the so-called Tisserand parameter *T* with respect to Jupiter (*JUP*) that depends on the semi-major axis *a*, inclination *i* and eccentricity *e* of a comet. The resulting parameter has no unit and a value between 2 and 3 indicates a close encounter with Jupiter in the past. The comet became a Jupiter-Family Comet (JFC).

$$T_{\mathrm{Jup}} = \frac{a_{\mathrm{Jup}}}{a} + 2\cos(i)\sqrt{\frac{a}{a_{\mathrm{Jup}}}(1-e^2)}$$

Imagine using and describing a multi-input-parameter equation in a thesis or research paper (or in a Medium article). The equations, results and plots are trivial to you, who works in this particular research field all the time. But scientific insights and results need to be explainable and accessible for others. Otherwise, you cannot convince others of your ideas. Things stay too complex and your ideas cannot thrive. Every day dozens of research papers are being published. *What is good? What is worth reading? Shall I believe what the researcher claims?*

Plots, tables and images help us and the reader to understand the complexity of research results and creating interesting plots can become a craft. But there are limitations:

1. Some research activities create tons of data that cannot be printed or displayed
2. A research paper or thesis is “2 dimensional”

Both issues can be easily handled with an additional online section for *Supplementary Materials*. For example, *Science* provides a guideline for these materials that cannot be published in the print version of the paper. Code can be pushed on GitHub, data can be stored in the Cloud and multimedia information can help the reader to improve the understanding of the published content.

In our case: We have a function that depends on 3 input parameters. *Do you have an idea how the solution space of the Tisserand parameter looks like?* … me neither. So let’s improve our understanding of the Tisserand parameter by creating an animation that covers all input parameters.

Python’s support

For this lesson, first, we need our already known libraries. Later new modules are introduced for new purposes and features. *pathlib* will be needed for creating a folder structure (line 1); we have again *spiceypy* for computing the semi-major axis of Jupiter (line 5); line 8 *imports* the meta kernel file with all required kernel paths and *numpy* (line 11) as well as *matplotlib* (line 14) are needed for data handling and plotting, respectively.

Part 1/7

For the Tisserand Parameter we need the semi-major axis of Jupiter. We set an Ephemeris time (ET) in line 2 by converting a UTC string to ET (with *utc2et*). Then (line 6 to 9), we determine the Jupiter barycentre (NAIF ID: 5) state vector in ECLIPJ2000 as seen from the Sun (NAIF ID: 10) with the function *spkgeo*. After extracting the G*M (Gravitational constant times the mass) value of the Sun in line 12 and 13 (with the function *bodvcd*), we determine Jupiter's orbital elements in line 16. The function *oscltx* requires the state vector of Jupiter, the ET and the G*M value of the Sun as inputs. The second last entry of the resulting array *ORB_ELEM_JUPITER* contains the semi-major axis in km (line 19). Line 20 converts the value from km to AU using the SPICE function *convrt*.

Part 2/7

How can we visualise the Tisserand Parameter that requires 3 different input parameters? One suggestion: We create a 2 dimensional plot with the eccentricity on the x and the inclination on the y axis. A colour contour map represents the Tisserand value. Then, we can create several colour contour plots for different semi-major axis values. The resulting images are then merged into a movie or animated GIF.

We will use the *maplotlib contourf function* to create these plots. First, with *numpy meshgrid* (line 17) we set a grid of inclinations (line 3 and 4 define the inclination in degrees and radians) and eccentricities (line 7). Line 12 to 14 define the Tisserand parameter as a lambda function.

Part 3/7

Now we set some general formatting for the upcoming plots. For the colour contour plots we need some colour mapping. Thus, we import *matplotlib cm* in line 2. The dark mode plots look quite nice so we set the *dark_background* layout in line 5 and increase in line 8 the default font size for better readability.

Each contour plot will represent the solution space for a particular semi-major axis. The range of these values is defined at the end in line 12 and 13. An animation shall be created from 1.0 AU to 8.0 AU with a step size of 0.1 AU.

Part 4/7

Now we create the contour plots for each semi-major axis step. All plots shall be stored in an extra folder to prevent file flooding in our main tutorial folder. Line 2 creates the folder *temp/* if the directory is not present yet. In total 71 images will be created. This may take some time and printing each step in the upcoming for-loop would spam the terminal output. To get a clean and dynamic working status we install the module *tqdm* that enables one to create great progress bars in the terminal or in Jupyter Notebook:

```
pip3 install tqdm
```

Line 5 imports *tqdm* that is being used in the for-loop statement in line 12. Below the code, you can see the resulting progress bar. Line 8 creates an initial figure that is used in the following loop.

The for-loop iterates through the semi-major axis values (*A_ARRAY*). First, the plot is cleared (line 15), the x and y limits are set (line 18 and 19) and the semi-major axis, eccentricity and inclination corresponding Tisserand parameter matrix is computed (line 22). After setting the title (that shows the semi-major axis) and labels (line 25–27) the contour plot is created (line 30–33). We want to distinguish clearly between Tisserand values smaller than 2, between 2 and 3 (JFCs) and larger than 3. Thus we set limits with *vmin* and *vmax* for the colour mapping. The values between 2 and 3 are separated in 10 steps (end line 32).

Afterwards, a colour bar is plotted with a description (line 35 and 36) and the resulting image is stored in the *temp/* folder. A dpi value of 100 has been chosen, since the final GIF shall not exceed more than 25 MB (maximum file size on Medium).

Part 5/7

```
In [5]:
```

tqdm output of the for-loop. Credit: T. Albin

Matplotlib provides a native animation support. However, we have chosen an approach to store all plots first. That way, one can re-use individual plots for other purposes. To merge the plots into one GIF we need to define an array that contains the order of the plots' paths. First, we import the library *glob* that allows us to obtain a list of paths of the created plots (line 4 and 5). In line 13 a list is created that contains the ascending order of paths. We add a short break into the animation by adding 25 times the last image (line 16). An inverted list of the paths' name is added to create a "back and forth" animation with another break of 25 images of the very first image (line 20 and 23).

Part 6/7

With the library *imageio* we can now create the GIF. If you have not installed this library yet, execute the following pip command to install it:

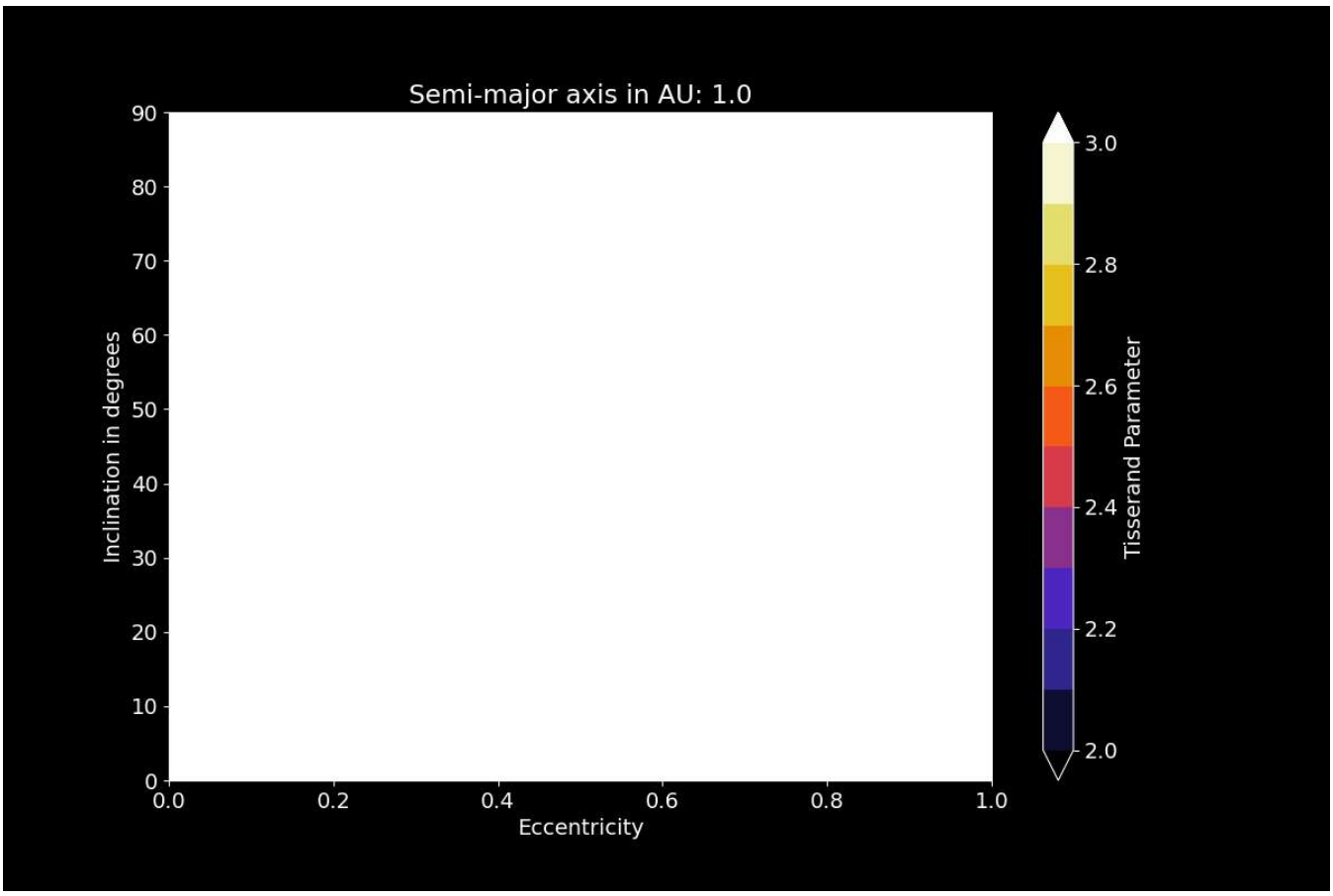
```
pip3 install imageio
```

Import the library (line 2) and create an empty list (line 5) that will contain all plot information. A for-loop in line 9 to 10 iterates through all path names and appends the read images (with *imageio imread*) to the originally empty list.

Now, in line 14 the animated GIF is being created with the function *mimsave*. The duration of each image is set to 0.04 s, resulting in an animation of 25 frames per second.

Part 7/7

Below you can see the resulting GIF of the Tisserand parameter solution space as a function of the semi-major axis, inclination and eccentricity. These kinds of animations help to grasp complex functions.



Animation of the Tisserand parameter as function of the semi-major axis, inclination and eccentricity. The colour map represents the Tisserand value that is displayed on the right hand side. The coloured (non-white, non-black) area represents the solution space of Jupiter-Family Comets (JFCs). Credit: T. Albin

Conclusion & Next step

Last time we learned about the Tisserand parameter and its interpretation (a value between 2 and 3 indicates a dynamical link between a comet and Jupiter). The function, however, is multi-dimensional and difficult to grasp.

Supplementary materials, either here, or for scientific papers, support the reader (and sometimes oneself!) to *get a feeling* of complex functionalities and dependencies.

It may take some time and effort to create a simple animation (like the one shown above), but a *picture is worth a thousand words* and animation probably more.

Next time we will get back on the science track and have a look at an important data scientific issue: Bias effects.

Thomas

References

[1] Tisserand, F.-F. (1896). *Traité de Mécanique Céleste IV*. Gauthier-Villars, Paris

Python Space Science Programming Space Science With Python

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter

About Help Terms Privacy

Get the Medium app

Download on the App Store

GET IT ON Google Play